

EE5203 Microprocessors Homework: ADC & PWM

EE5203 Microprocessors

Basri ERDOGAN

2026-07-23

Instructions

Record your homework in a video and share the link in your submission and do not forget to attach your main.c file.

TASK 1: ADC & LED Bar Graph

Objective:

Read an analog voltage from a potentiometer and visualize the voltage level using 8 LEDs, similar to a VU meter.

Steps:

1. **Hardware Setup:** Connect a potentiometer to providing 0-3.3V to the ADC input (PA0). Connect 8 LEDs to suitable GPIO pins.
2. **ADC Configuration:** Configure the ADC to read the analog voltage.
3. **Logic Implementation:**
 - Read the converted digital value.
 - Divide the 3.3V range into 8 segments (approx. 0.41V per segment).
 - Turn on the specific LED corresponding to the current voltage level.
4. **Verification:** Rotating the potentiometer should light up different LEDs in sequence.

Application Notes:

- **Triggering:** Use **Timer2** to trigger the ADC conversion (sample rate: 1000 times per second).
- **ADC Settings:**
 - *ADC_Regular_ConversionMode -> External Trigger Conversion*
Source = Timer2 Trigger Out event.
 - Enable **Continuous conversion mode**.

- **Pinout:** Use **PA0** for the analog input. Choose any 8 free GPIO pins for the LEDs.

Component List:

1x STM32F401, 8x LEDs, 1x potentiometer, 1x multimeter, jumper cables

TASK 2: PWM Brightness Control

Objective:

Use Pulse Width Modulation (PWM) to smoothly increase the brightness of an LED.

Steps:

1. **Timer Configuration:** Use STM32CubeMX to configure **Timer2 Channel 1** for PWM generation.
2. **Hardware:** Connect an LED to the corresponding GPIO pin for Timer2 Channel 1.
3. **Code Implementation:** Write a loop that gradually increases PWM duty cycle with a button press.

[Reference Video Link](#)

Application Notes:

- The brightness of the LED is determined by the **Duty Cycle** of the PWM signal.
- Modify the TIM2->CCR1 register (or use `__HAL_TIM_SET_COMPARE`) to change the duty cycle dynamically.
- Read input from a button

TASK 3: Advanced UART Control

Objective:

Implement a two-way serial communication system. The STM32 will report the **potentiometer voltage (from Task 1)** and the current LED brightness to the PC. Simultaneously, the PC will send commands to override/control the LED brightness (PWM) from Task 2.

Steps:

1. **Configure USART2:** Set up USART2 for asynchronous communication at 115200 baud. Enable the **USART2 global interrupt** in the NVIC settings.
2. **Status Updates (TX):** Every 1 second, send a formatted string to the PC showing the current **potentiometer voltage (from Task 1)** and the current PWM duty cycle (e.g., "Voltage: 1.25V | PWM: 40%\r\n").
3. **Command Processing (RX):**
 - Implement `HAL_UART_RxCpltCallback` to receive single characters from the PC.
 - If the user presses '+', increase the LED brightness (PWM duty cycle) by 10%.
 - If the user presses '-', decrease the LED brightness by 10%.
 - If the user presses '0', turn the LED off (0% duty cycle).
 - If the user presses '1', turn the LED fully on (100% duty cycle).
4. **Feedback:** Immediately after receiving a valid command, send a confirmation message back to the PC (e.g., "Brightness increased to 50%\r\n").

Application Notes:

- Use `sprintf` to format your status strings before transmitting.
- Use `HAL_UART_Receive_IT` to listen for incoming characters without blocking your main loop.
- Ensure you handle the PWM duty cycle limits (0% to 100%) correctly so it doesn't wrap around.